

Aspect-Level Inheritance Verification: Operationalizing the Recursive Application of Series A's Test Suite at Aspect Scope in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 8, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Note ID: B2.19 (Phase B2, closing the B1.04 decomposition)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in *The Instinct/Reasoning Separation Outside the Model* (Li, April 2026), the second paper in the CKS theory series following *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize how the sixteen Series A operational tests A5.01–A5.16 are applied at aspect scope to verify that Paper 1's commitments hold operationally for aspects per the recursive inheritance commitment of B1.20, paralleling cell-level inheritance verification per B2.14, and closing the five-note B1.04 decomposition.

Abstract

Paper 2 commits to recursive inheritance: Paper 1's six architectural commitments hold at every structural level — cell, aspect, and Self — without modification (B1.20). For this commitment to be operationally meaningful rather than rhetorical, a deployment must exhibit Paper 1's properties at each level under a verification mechanism the architecture commits to. Series A's sixteen operational tests A5.01–A5.16 are the canonical mechanism at cell scope; B2.14 formalizes their cell-level application. This note formalizes their aspect-level application as **aspect-level inheritance verification**: the same sixteen test specifications, run on aspect-level architectural artifacts (aspect purpose specifications, coordination rules, membership records, multi-aspect cell participation arrangements) under aspect-distinct semantics. The standardization of the test suite across levels is what makes recursive inheritance operationally demonstrable. Aspect-level verification is governed per A1.01, recorded per A2.40, runs concurrently with cell-level verification per B2.14, and operationally gates aspect birth, modification, mating, and vertical evolution. This note is the fifth and closing element of the B1.04 (aspect as coordination arrangement) decomposition.

1. Why aspect-level inheritance verification needs to be formalized as standalone

Paper 2's architecture introduces three structural levels — cell, aspect, and Self — and commits Paper 1's commitments recursively at each level. A separate note (B1.20) formalizes the recursive inheritance at the foundational level. Two further notes formalize how that inheritance is operationally verified: B2.14 at cell level, and this note at aspect level.

Recursive inheritance is a strong architectural claim: it asserts that humans retain inspect, modify, and override authority over aspect-level coordination, that aspect operations preserve provenance, that aspect behavior is deterministic given inputs, and so on. A claim of this magnitude requires a corresponding operational mechanism, or it is rhetoric. Series A's sixteen operational tests A5.01–A5.16 are the canonical mechanism Paper 1 commits to at cell scope. What this note formalizes is that the same test specifications, applied at aspect-level architectural artifacts under aspect-distinct semantics, are the canonical mechanism for aspect-level inheritance verification.

This note is the fifth and closing element of the B1.04 decomposition (aspect as coordination arrangement), following B2.15 (purpose-defined coordination), B2.16 (coordination rules), B2.17 (content-domain operationalization), and B2.18 (multi-aspect cell participation). With this note, the operational machinery of aspect-level inheritance verification is in place, and the B1.04 decomposition closes.

2. Aspect-level inheritance verification, defined precisely

A deployment instantiates aspect-level inheritance verification when, for each aspect within scope, the sixteen Series A operational tests A5.01–A5.16 are applied at aspect-level architectural artifacts under aspect-distinct semantics, are governed by humans per A1.01, and have their results recorded as substrate content per A2.40. The sixteen tests, applied at aspect level:

A5.01 — Inspect right test. Verifies that humans with appropriate authority can read aspect-level substrate content directly per A2.01: aspect purpose specification, coordination rules, membership records, aspect-level outputs.

A5.02 — Modify right test. Verifies that humans can edit aspect-level orchestration content per A2.02: aspect purpose, coordination rules, membership rules.

A5.03 — Override right test. Verifies that humans can override specific aspect-level decisions or membership outcomes per A2.03 without procedural justification.

A5.04 — Rule authoring test. Verifies that aspect behavior is governed by human-authored rules per A2.04.

A5.05 — Mediator role test. Verifies that LLM consultation by aspect operations preserves Properties A–E (A2.18–A2.23) when aspect operations consult the instinct layer per B2.01.

A5.06 — Aspect-behavior-determinism test. Adapts the cell-behavior-determinism test to aspect scope. Verifies that aspect behavior given inputs is deterministic.

A5.07 — Read-determinism test. Verifies that reads of aspect-level substrate return consistent results.

A5.08 — Provenance-completeness test. Verifies that aspect operations carry complete provenance per A2.40.

A5.09 — Four-accountability-questions test. Verifies that the substrate answers who, what, when, why for every aspect operation.

A5.10 — Source-of-truth-five-categories test. Verifies that aspect content fits the five categories A2.42–A2.46.

A5.11 — Tool-agnosticism-migration test. Verifies that the aspect operates correctly across substrate-technology migration per A1.05.

A5.12 — Linear-cost-scaling test. Verifies that aspect governance cost scales linearly per A1.06.

A5.13 — Conflict-coexistence test. Verifies that the aspect handles cross-cell conflicts per A1.03 — preserving conflicting cell outputs as first-class substrate state, surfacing them through aspect coordination rules per B2.16, rather than silently merging them.

A5.14 — Composition-requirements-five test. Verifies that aspect-cell composition meets the five composition requirements A1.13.

A5.15 — Pattern-mapping test. Verifies that aspect-cell relationships fit Pattern A, B, or C per A2.92–A2.94.

A5.16 — Reproducibility test. Verifies that aspect operations replay deterministically.

The pattern is consistent: same test specifications as Series A defines, applied at aspect-level artifacts. The architectural artifacts under test differ from those at cell level — at aspect level, A5.01 inspects aspect purpose specifications and coordination rules, not individual cell substrate content. This is what aspect-distinct semantics names: the test logic recurs; the targets shift to aspect-level artifacts.

This is also what makes recursive inheritance per B1.20 operationally demonstrable rather than merely claimed. With a consistent test suite — the same sixteen specifications, run at cell, aspect, and Self artifacts — the recursive-inheritance assertion becomes a property the deployment exhibits or fails to exhibit. Verification is governed per A1.01: humans configure which tests run, at what cadence, with what parameters, and review the results. Verification events are recorded per A2.40: the verification record is itself substrate content, subject to the same authority architecture as the artifacts under test.

3. What makes aspect-level inheritance verification architecturally distinctive

Conventional AI architectures often verify component-specific properties without an architectural test suite at multiple levels. A particular module is unit-tested for its specific function; the whole system may be integration-tested for end-to-end behavior; but the same architectural property — say, “humans can inspect the system” — is not tested in a structurally identical way at module, subsystem, and system scope. Testing is component-level only, or aggregate-level only. This is a different testing posture from what CKS deployments require, not an indictment of those architectures.

CKS deployments commit to recursive Paper 1 commitments at every structural level (B1.20). The cell, the aspect, and the Self each carry the same architectural properties. To make this commitment operationally meaningful, the same test specifications run at each level. The recursion of the test suite is what demonstrates the recursion of the commitments.

Aspect-level verification carries one specifically distinctive scope: aspect-level architectural artifacts that have no precise cell-level analog. Aspect purpose specifications, aspect coordination rules, aspect membership records, and multi-aspect cell participation arrangements (B2.18) are aspect-level artifacts. Cell-level inspection (A5.01 at cell scope) does not cover them. Without aspect-level verification, these artifacts would either be unverified, leaving a gap in the recursive inheritance claim, or verified ad-hoc by

deployment-specific mechanisms outside the architectural framework. Aspect-level inheritance verification closes this gap by applying the standard test suite to aspect-level artifacts under aspect-distinct semantics.

The architectural distinctiveness is not the existence of testing — many architectures test their components — but the specific posture of the same test specifications applied at multiple levels with level-distinct semantics. This posture is what makes a recursive inheritance claim demonstrable rather than asserted.

4. The cognitive analog

Aspect-level inheritance verification has a loose cognitive analog in system-level testing within modular software engineering: the same test specifications can apply at unit, integration, and system scope, with the artifacts under test shifting at each level. A unit-level access-control test verifies one module's authorization checks; an integration-level version verifies cross-module authorization; a system-level version verifies end-to-end authorization. The test logic is structurally similar; the targets differ. Biology offers a looser analog in the distinction between cell-level cytopathology and tissue-level histopathology: related diagnostic logic, different level of organization.

Both analogs function as conceptual scaffold. They make the recursive-test-suite posture intuitive to readers familiar with modular software or with biology. Neither analog supplies the architectural substance, which is the recursive application of CKS's specific sixteen-test suite at aspect scope under aspect-distinct semantics. The conceptual scaffold helps readers absorb the shape; the architectural work is CKS's own.

5. Inherited Paper 1 commitments

Aspect-level inheritance verification inherits Paper 1's commitments structurally, since the verification itself is governed by them.

A1.01 (human-governed). Verification is configured and reviewed by humans. Which tests run, at what parameters, on which aspects, with what frequency — all are decisions held under the inspect, modify, and override rights.

A1.07 (path retraceability). Verification events are recorded with full provenance — who ran the test, what it covered, when, why.

A1.10 (determinism contract). Verification tests are themselves deterministic: running the same test against the same substrate state yields the same result.

A1.13 (composition requirements). A5.14 explicitly tests the five composition requirements at aspect scope.

A2.04 (rule authoring). The rules that govern verification — when tests run, what counts as passing, what action a failure triggers — are themselves authored under human authority.

A2.40 (provenance metadata). Verification events are substrate content with the standard six-field provenance.

A5.01–A5.16 (operational tests). The test specifications are inherited verbatim. Aspect-level inheritance verification does not author new tests; it applies Series A's tests at aspect scope.

B1.20 (recursive inheritance). This is the foundational claim aspect-level verification operationally establishes.

A1.03 (conflict as first-class). A5.13 at aspect scope verifies that aspect coordination rules per B2.16 surface and preserve conflicting cell outputs rather than auto-resolving them. The conflict commitment is inherited and tested at aspect level.

6. Operational implications

Several operational consequences follow.

Verification configuration is per aspect type. Different aspect types may carry different test parameter values for the same standard suite. A regulated-review aspect may require tighter A5.06 determinism tolerances than an exploratory-analysis aspect. The architecture supplies the test; the deployment supplies the parameters under A1.01.

Verification gates aspect lifecycle. A newly instantiated aspect must pass aspect-level verification at birth (B1.09) before being treated as operational. A modified aspect must re-verify after changes to its purpose, coordination rules, or membership. An aspect that has undergone vertical evolution per B1.16 must re-verify at the new structural form. Aspects that fail and cannot be brought into compliance may be subject to functional obsolescence death per B1.11.

Verification interacts with mating. Aspects created through mating (B1.10) — selective merge, lineage-preserved union, or full union of parent aspects — must pass aspect-level verification before operating. Mating that produces an aspect failing the conflict-coexistence test (A5.13 at aspect scope) reveals that the merge has silently resolved cross-cell conflicts rather than preserving them; such a merge is invalid under A1.03.

Cross-partner verification follows authority distribution. When aspect-level verification spans authority boundaries, verification is run per the cross-partner authority distribution per A2.47. Each authority verifies what falls within its scope; the substrate records the verification provenance per partner.

Aspect-level verification operates concurrently with cell-level verification. Per B2.14, cell-level inheritance verification is also run; both are required for full inheritance assurance. Aspect-level verification cannot substitute for cell-level: an aspect whose constituent cells fail cell-level verification cannot pass aspect-level verification merely by satisfying aspect-level artifacts. Both levels run; together they cover the level scopes per B2.07.

Verification failures trigger governance actions. A failure of A5.01 at aspect level means human inspection is somehow blocked for an aspect-level artifact; the response is governance action — direct override per A2.03, corrective rule authoring per A2.04, or escalation per the deployment's governance procedures. Verification is not a passive monitor; it is an operational gate.

7. Limits

The standalone treatment of aspect-level inheritance verification is bounded.

It does not replace cell-level or Self-level verification. Distinct levels carry distinct verification scope. Aspect-level verification covers aspect-level artifacts; cell-level verification per B2.14 covers cell-level artifacts; Self-level verification (a subsequent Phase B2 variant) will cover Self-level artifacts. The three are concurrent and complementary, not hierarchical and substitutable.

It does not eliminate domain-specific aspect testing. Inheritance verification covers architectural inheritance — that Paper 1's commitments hold at aspect level. Whether an aspect performs its domain function correctly is a separate testing concern that remains a deployment responsibility.

It does not prescribe specific test parameters. Deployment teams configure parameters per aspect type under A1.01. The note specifies which tests apply at aspect level; it does not specify thresholds, frequencies, or values.

It does not guarantee behavioral correctness. The sixteen tests verify architectural properties. An aspect can pass all sixteen and still produce incorrect domain outputs. Inheritance verification is necessary, not sufficient, for operational soundness.

It is not a single test but the application of A5.01–A5.16 at aspect scope. Treating it as a single composite obscures the operational structure. Each of the sixteen carries independent operational content; failure of one is not equivalent to failure of another.

It does not eliminate verification gates for instinct. Per B2.06, instinct-layer (LLM) behavior is verified through distinct mechanisms — verification substrates, parallel-run detection, and so on. Aspect-level inheritance verification covers aspect-level reasoning artifacts; instinct verification covers a different scope.

It does not substitute for cell-level verification of constituent cells. Per B2.14, cell-level inheritance verification covers constituent cells. A deployment that verifies aspects but skips cells, or verifies cells but skips aspects, has not operationally demonstrated B1.20 in full.

8. Operational test

A deployment instantiates aspect-level inheritance verification if and only if all of the following are true for every aspect within scope:

1. The sixteen Series A operational tests A5.01–A5.16 are applicable at aspect-level architectural artifacts under aspect-distinct semantics.
2. The tests are run on the aspect at aspect birth (B1.09), at modification of aspect purpose, coordination rules, or membership, and at vertical evolution (B1.16).
3. Test parameters are configured by humans under A1.01 governance, and the configuration is itself substrate content with provenance per A2.40.
4. Verification results are recorded as substrate content with provenance per A2.40, and the verification records are themselves subject to A1.01 governance (inspectable, modifiable, overridable).
5. Verification failures trigger governance actions per A2.03, A2.04, or B1.11 — rather than being absorbed silently or routed into non-architectural workflow.

6. Aspect-level verification operates concurrently with cell-level verification per B2.14 and does not substitute for it; constituent cells must independently pass cell-level inheritance verification.

A deployment that fails any of (1)–(6) does not instantiate aspect-level inheritance verification. Such a deployment may carry other testing posture, and may even be useful, but it does not operationally demonstrate the recursive inheritance commitment of B1.20 at aspect level.

9. Conclusion

Naming aspect-level inheritance verification as standalone formalizes the operational mechanism by which Paper 2's recursive inheritance claim becomes demonstrable at aspect level. Without it, the claim that Paper 1's commitments hold at aspect scope is rhetoric — a proposition the architecture asserts but does not exhibit. With it, deployments either pass aspect-level verification or they do not, and recursive inheritance becomes a property the deployment can be audited against rather than a posture it can claim without test.

Aspect-level verification is operationally consequential. It gates aspect birth, modification, mating, and vertical evolution; it runs concurrently with cell-level verification per B2.14; and it specifically covers aspect-level artifacts that have no cell-level analog — purpose specifications, coordination rules, multi-aspect cell participation arrangements. The recursion of the test suite across levels — same sixteen specifications applied at cell, aspect, and forthcoming Self artifacts — is what makes recursive inheritance per B1.20 architecturally substantive rather than merely claimed.

This note closes the B1.04 (aspect as coordination arrangement) decomposition. The five notes B2.15, B2.16, B2.17, B2.18, and B2.19 (this note) together formalize the operational structure of aspect as coordination arrangement. Phase B2 continues with B2.20 beginning the B1.05 Self-level decomposition, which follows a parallel structure and culminates in Self-level inheritance verification as the operational mechanism establishing recursive inheritance at Self scope.

Subsequent work that adopts, extends, or argues against the CKS inheritance-verification posture at aspect level should use “aspect-level inheritance verification” in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Aspect-Level Inheritance Verification: Operationalizing the Recursive Application of Series A's Test Suite at Aspect Scope in the Coordination Knowledge Substrate Pattern*. May 8, 2026. ORCID: 0009-0004-8065-3235.